

Enhancing vulnerability detection efficiency: An exploration of light-weight LLMs with hybrid code features

Jianing Liu ^{a,c,d}, Guanjun Lin ^b, Huan Mei ^{a,c,d}, Fan Yang ^{e,*}, Yonghang Tai ^{a,c,d,*}

^a School of Physics and Electronic Information, Yunnan Normal University, Kunming, 650000, China

^b School of Information Engineering, Sanming University, Sanming, Fujian, 365004, China

^c Engineering Research Center of Photoelectric Detection and Perception Technology, Yunnan Normal University, Kunming, 650500, China

^d Yunnan Key Laboratory of Optoelectronic Information Technology, Kunming, 650500, China

^e Yunnan Normal University Institute of Physical Education, Kunming, 650500, China

ARTICLE INFO

Keywords:

Vulnerability discovery
Deep learning
Source code

ABSTRACT

Vulnerability detection is a critical research topic. However, the performance of existing neural network-based approaches requires further improvement. The emergence of large language models (LLMs) has demonstrated their superior performance in natural language processing (NLP) compared to conventional neural architectures, motivating researchers to apply LLMs for vulnerability detection. This paper focuses on evaluating the performance of various Transformer-based LLMs for source-code-level vulnerability detection. We propose a framework named VulACLLM (AST & CFG-based LLMs Vulnerability Detection), which leverages combined feature sets derived from abstract Syntax Tree (AST) and Control Flow Graph (CFG). The recall rate of VulACLLM in the field of vulnerability detection reached 0.73, while the F1-score achieved 0.725. Experimental results show that the proposed feature sets significantly enhance detection performance. To further improve the efficiency of LLM-based detection, we examine the performance of LLMs compressed using two techniques: Knowledge Distillation (KD) and Low-Rank Adaptation (LoRA). To assess the performance of these compressed models, we introduce efficiency metrics that quantify both performance loss and efficiency gains achieved through compression. Our findings reveal that, compared to KD, LLMs compressed with LoRA achieve higher recall, achieving a maximum recall rate of 0.82, while substantially reducing training time, taking only 20 min to complete one epoch, and disk size, requiring only 4.89 MB of memory. The experimental results demonstrate that LoRA compression effectively mitigates deployment challenges associated with large model sizes and high video memory consumption, enabling the deployment of LoRA-compressed LLMs on consumer-level GPUs without compromising vulnerability detection performance.

1. Introduction

To date, software programs have become increasingly vital, being an indispensable part of nearly every industry. However, as the scope of software usage expands, the likelihood of security vulnerabilities in applications also increases, making them more susceptible to cyberattacks [1]. Recent security incidents involving companies such as Microsoft, Google, and Apple have become more frequent [2]. Domestically and internationally, many major cyberattacks — such as ransomware incidents — have been triggered by software vulnerabilities [3]. These vulnerabilities not only threaten real-world systems by paralyzing critical software, but they can also lead to the leakage of sensitive personal information and even jeopardize national security. For instance, the recent Log4j vulnerability compromised systems within the U.S. federal government, while the Russian Sandworm

group exploited the Follina vulnerability to infiltrate key institutions in Ukraine [4,5].

With the exponential growth in the scale and complexity of applications, as well as an increasing number of potential attack surfaces, manually inspecting source code for vulnerabilities has become impractical. However, advancements in computational resources, along with ongoing research in artificial intelligence, have driven rapid developments in deep learning. The evolution of model methodologies has progressed from traditional machine learning algorithms — such as Support Vector Machine (SVM) — to large-scale language models (LLMs) containing billions of parameters. Prominent examples of such models include GPT [6], BERT [7], XLNet [8], and LLaMA [9]. These advancements have transformed fields like computer vision and natural

* Corresponding authors.

E-mail addresses: 610204995@qq.com (F. Yang), taiyonghang@126.com (Y. Tai).

<https://doi.org/10.1016/j.jisa.2024.103925>

language processing, inspiring cybersecurity researchers to leverage LLMs for identifying system vulnerabilities [10].

In addition to using LLMs for detecting vulnerabilities in source code, researchers are investigating methods for decomposing code into various representations, such as Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Program Dependence Graphs (PDG). Once these representations are extracted, deep learning techniques — including Convolutional Neural Networks (CNN) [11], Long Short-Term Memory (LSTM) networks [12], and Bidirectional LSTM (BiLSTM) — are employed to process these code representation graphs [13]. Using such representations allows models to capture the intricate details of source code, conveyed through syntax, data flow, and control flow, thereby enabling LLMs to better identify patterns indicative of potential vulnerabilities [14,15].

While LLMs have significantly improved performance across various research areas, the exponential growth in model parameters has also led to higher hardware demands. Many tasks involving LLMs now require professional-grade GPUs for optimal performance. The gap in performance between consumer-grade and professional-grade GPUs is substantial, particularly when training large models. Additionally, the limited video memory of consumer-grade GPUs often prevents them from supporting LLM-based tasks effectively. For cybersecurity companies focused on vulnerability detection, LLMs offer superior performance compared to traditional deep learning models [10]. However, professional-grade GPUs are significantly more expensive than consumer-grade GPUs and also consume more energy. Although LLMs have demonstrated improvements in detecting vulnerabilities in source code, the substantial investment required to deploy these models limits their practical use.

For the above issues, this paper proposes the following contributions: (To ensure reproducibility, we have publicized our code and partial data at: <https://github.com/JNL-28/LLMs-C>.)

- Compare and select mainstream LLMs, and identify the most effective LLMs in the field of source-code level vulnerability detection.
- Based on the characteristics of the AST and the CFG, a hybrid approach combining the structured features of AST and CFG representations, termed VulACLLM (AST & CFG-based LLMs Vulnerability Detection), is proposed. This method aims to investigate the feasibility of leveraging these hybrid code representations as features along with LLMs in the domain of source-code vulnerability detection.
- We aim to explore performance solutions in the field of source-code vulnerability detection to assess the feasibility of applying LLMs with improved efficiency. Therefore, we compared the model compression approaches using LoRA and knowledge distillation to evaluate their differences in performance in source-code vulnerability detection. This comparison led us to propose an efficient vulnerability detection framework that balances the efficiency of LLM training and detection performance.
- To evaluate the performance of compressed LLMs and explore how to select LLMs within the hybrid framework (VulACLLM) that achieve a balance between performance and efficiency, we propose the Top-K Efficiency metric. This metric quantifies the performance loss and efficiency gains resulting from model compression.

The rest of the paper is organized as follows: Part II reviews existing work and model methodologies in natural language processing and explores the models and research methods used in vulnerability detection. Part III presents the background and motivation for applying compact LLMs to source code vulnerability detection. Part IV provides a detailed description of the experimental setup and procedure for evaluating the performance of LLM compression solutions for detecting vulnerabilities. Part V summarizes the entire paper and offers prospects for future work.

2. Related work

The significant advancements achieved in natural language processing (NLP) through deep learning have intensified the interest of cybersecurity professionals in applying deep learning language models to vulnerability detection. Before LLMs like GPT, BERT, and others gained widespread usage, researchers optimized the weights of multilayer perceptron (MLP) networks using algorithms to enhance the accuracy of network intrusion detection systems in detecting traffic anomalies [16]. Another study involved using SVM as classifiers to extract N-gram features from web-based malicious code for the classification of malicious or benign code. Additionally, researchers employed SVM for the malicious identification of Android applications, achieving promising results [17,18].

The methods utilized by researchers in NLP have also inspired cybersecurity practitioners. Vectorizing source code similarly to image and natural language processing represents a crucial approach for vulnerability detection in cybersecurity. CNN has maintained popularity since their inception, with researchers across various industries innovating based on them. For instance, Jung Hyun An et al. proposed V-CNN (Vulnerable Convolutional Neural Network), which employs CNN to learn from Common Weakness Enumeration (CWE) / Common Vulnerabilities & Exposures (CVE) datasets for vulnerability detection [19]. Another study combines CNN and LSTM to create a composite neural network model known as VulExplore, which is utilized for vulnerability detection [20]. Additionally, researchers have combined CNN and Recurrent Neural Network (RNN) to extract diverse features, achieving state-of-the-art results on the SARD dataset [21].

Numerous research endeavors focus on various neural network models for vulnerability detection. However, since the advent of Transformers [22], many variants based on this model have emerged. A notable example is BERT (Bidirectional Encoder Representations from Transformer), a pretrained model utilizing a multi-layer Transformer architecture. BERT is constructed by stacking multiple Transformer encoder layers sequentially. Furthermore, BERT has inspired the development of numerous effective models. For instance, Soolin Kim et al. proposed VulDeBERT, a new vulnerability detection tool based on BERT, which surpasses VulDeePecker in detecting two types of code vulnerabilities (CWE-119 and CWE-399) [23]. Weichang Huang et al. introduced BBVD (BERT-based Method for Vulnerability Detection), a method leveraging BERT to treat high-level programming languages as natural language for vulnerability detection. This approach demonstrates strong performance on the SARD and Big-Vul datasets [24].

Another widely popular variant based on Transformers is GPT (Generative Pre-trained Transformer). GPT shares similarities with BERT; However, GPT is a model composed of multiple Transformer decoder layers. David Noever and colleagues evaluated the capability of the LLMs GPT-4 in detecting software vulnerabilities. They found that GPT4 discovered approximately four times as many vulnerabilities as similar detectors [25].

Another approach to vulnerability detection in source code involves extracting code representation graphs such as AST, CFG, and PDG. Zhenzhou Tian and colleagues decompose the source code into AST fragments, which are then transformed into deep, finite, and ordered subtrees. This process enables the neural network to capture the semantics of each subtree, thereby improving accuracy [26]. Minmin Zhou and colleagues proposed a method based on an improved control flow graph (ICFG). This approach achieved good detection results for three specific types of vulnerabilities: Unused_Variable, Use_of_Uninitialized_Variable, and Use_After_Free [27].

3. Background and motivation

This section briefly introduces the challenges faced by LLMs and discusses the characteristics and motivations behind some code representation graphs.

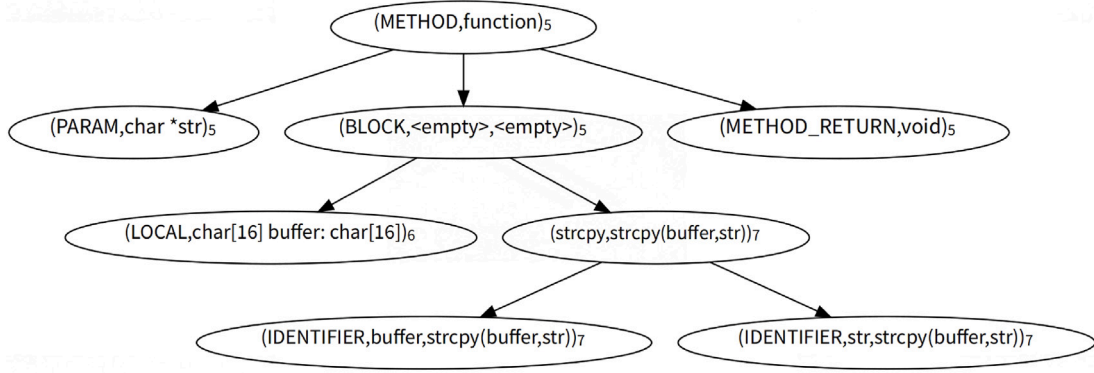


Fig. 1. A schematic diagram of the AST structure extracted using Joern.

3.1. LLMs: Challenges

LLMs excel in various NLP tasks due to their enormous size, containing billions of parameters, which results in a qualitative transformation stemming from quantitative changes.

Since Google introduced the Transformer model in 2017 and proposed the BERT model in 2018, the number of parameters in these models has increased from 340 million to 1 billion and 17 billion, respectively, culminating in GPT-3 with 175 billion parameters. After undergoing extensive learning with a vast number of parameters, the model has experienced a significant performance improvement and the emergence of certain capabilities, such as logical reasoning abilities (generally believed to manifest after reaching 10 billion parameters), the capacity to store massive amounts of knowledge, and enhanced contextual learning abilities. Meanwhile, the emergence of models continues to be propelled by both academia and industry, with the aim of achieving or surpassing existing state-of-the-art (SOTA) models. The scale of parameters is continuously expanding, and computational power has become remarkably substantial. This level of computational power surpasses the capabilities of most academic research groups and typical vulnerability detection software companies. With the exponential growth in parameter count, the high hardware demand constrains many applications utilizing LLMs. Additionally, the generalization ability of LLMs has not been fully realized.

On the other hand, academic research groups and general vulnerability detection software companies are compelled to employ more expensive hardware resources for research in the field of source code vulnerability detection, given the significant increase in model size and computational demand. This results in increased human, financial, and environmental costs associated with implementing downstream tasks using LLMs. Researchers have also taken note of this issue [28]. The increase in various costs poses significant challenges to the widespread application of LLMs in the field of source code vulnerability detection. Therefore, to control costs and enhance the performance of LLMs in the field of vulnerability detection, pre-trained models were selected for exploratory experiments.

3.2. AST & CFG: A hybrid structural feature sets

This section introduces and analyzes two types of code representation graphs used for vulnerability detection: AST and CFG. We used Joern to extract the AST and CFG structures.

3.2.1. AST

AST serves as a tree-like depiction of the abstract syntactic arrangement of the source code, with each node on the tree symbolizing a structure evident in the source code. Nonetheless, the AST does not portray the exact syntax, since nested parentheses are concealed within the tree structure and are not depicted as individual nodes. Its tree

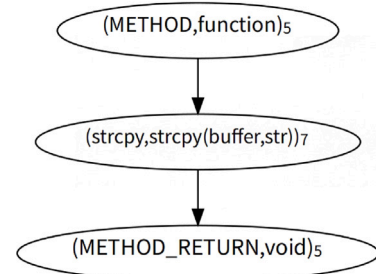


Fig. 2. A schematic diagram of the CFG structure extracted using Joern.

configuration offers a convenient method for grasping the arrangement and nested content of the source code. Simultaneously, the AST encapsulates both the syntax and structural data of the source code, abstracting certain details that do not impact the program's semantics. This capability allows the AST to be utilized in compilers and other utilities to maintain the semantics of the source code. Furthermore, the AST can be deployed in static code analysis and verification. By scrutinizing the tree structure, potential errors, network security vulnerabilities, or instances of subpar coding practices can be pinpointed. The schematic diagram of the AST structure is shown in Fig. 1.

3.2.2. CFG

CFG serves as a graphical depiction technique employed to illustrate the control flow within program code. It constitutes a directed graph wherein nodes symbolize basic blocks, and edges delineate the flow of control. A basic block encompasses a series of consecutive code segments whose execution adheres to the directed graph's sequence. This directed portrayal of code facilitates a deeper comprehension of the execution paths and control flow embedded within the source code. Furthermore, the CFG furnishes a visual mechanism for grasping the program's structure, enabling clear visualization of control flow constructs like branches, loops, and function invocations. Moreover, potential security vulnerabilities such as buffer overflows and code injections can be identified through analysis of the control flow. The schematic diagram of the CFG structure is shown in Fig. 2.

3.2.3. The hybrid approach

For certain datasets or types of vulnerabilities, using a single type of code representation graph for vulnerability detection predictions may yield satisfactory results [29]. However, for other datasets or types of vulnerabilities, the outcome may fall short of expectations [30].

CFG offers a lucid depiction of code execution flow, facilitating comprehension of program control structures and aiding in path analysis of the source code. However, CFG primarily emphasizes control flow, potentially overlooking intricate syntax and semantic details within

the source code, limiting their applicability for certain tasks. In contrast, AST captures the syntax structure of the source code, furnishing insights into its organization and semantics. As AST encapsulates detailed information such as variables, expressions, and statements, they prove valuable for numerous code comprehension and transformation endeavors. Furthermore, AST is language-agnostic, supporting a wide array of programming language tools. Several researchers, including Hantao Feng et al. have explored the benefits of AST [31]. To fully harness the syntax structure of program source codes, they proposed a data method based on AST for extracting all syntactic features and minimizing data redundancy.

However, since their focus is primarily on the syntax structure of the code and does not involve control flow, it may not be comprehensive enough for certain analysis tasks. Consequently, they often combine analysis with other graphs depending on the specific task to balance the demands of detail and complexity.

AST encompasses syntax and semantic features, while CFG contains control flow information related to the sequence of statements. Leveraging the characteristics of code representation graphs, we propose a hybrid approach that combines graphs containing syntax and semantic features from both AST and CFG. Due to the contextual learning ability emerging from LLMs, we investigate whether such hybrid features could enhance LLMs' vulnerability detection performance. Thus, we propose a source code vulnerability detection framework, VulACLLM (AST & CFG-based LLMs Vulnerability Detection). This framework integrates the features contained in hybrid features with LLMs for source code vulnerability detection. We will explore its effectiveness in the field of source code vulnerability detection.

3.3. Model compression methods: Knowledge distillation & LoRA

In order to address the challenges posed by the difficulty in training LLMs due to the excessive number of parameters and the increased hardware resource costs required for downstream tasks, efforts have been made to compress LLMs to reduce their size.

Knowledge distillation stands as a prevalent model compression method utilized across diverse domains within deep learning [32]. This strategy entails transferring knowledge from a sophisticated model (frequently denoted as the "teacher model") to a simplified counterpart (often referred to as the "student model"). Through this compression mechanism, the student model leverages the generalization capabilities of the teacher model, thereby enhancing its performance on unseen datasets. Furthermore, the student model typically boasts fewer parameters compared to the teacher model, facilitating swifter inference speeds in environments with constrained resources. This grants the student model the ability to attain performance levels akin to those of the teacher model, even in settings characterized by limited computational resources or reduced deployment requirements.

The Low-Rank Adaptation (LoRA) strategy emerges as a recent model compression approach [33]. Its concept entails introducing a bypass alongside the primary Pre-trained Language Model (PLM). By multiplying two matrices, A and B, the initial matrix, A, undertakes dimension reduction, while the subsequent matrix, B, manages dimension expansion, with the intermediate layer dimension set at r , thereby mimicking the intrinsic rank. During training, the parameters of the PLM are fixed, and only the dimension reduction and expansion matrices are trained. The input and output dimensions of the model remain unchanged, and during output, the product of BA is added to the parameters of the PLM. Additionally, matrix A is initialized with a random Gaussian distribution, while matrix B is initialized with a zero matrix, ensuring that at the beginning of training, this bypass matrix remains a zero matrix.

To summarize the LoRA model compression method operates by freezing the weights of the pre-trained model and injecting trainable rank decomposition matrices into each layer of the Transformer architecture. However, LoRA currently only supports the Linear, Embedding,

Conv2d, and kv layers within the architecture [33]. We fine-tuned the kv layers within the architecture and applied them to downstream tasks in vulnerability detection.

This paper systematically compares the effectiveness of LLMs compressed using knowledge distillation and LoRA methods within the VulACLLM framework. The aim is to achieve energy savings and efficiency improvements. Both knowledge distillation and LoRA compression methods effectively reduce training parameters and the demand for industrial-grade GPUs, thereby lowering the deployment costs of LLMs. Additionally, they address the issue raised in Section 3.2.3, where an excess of feature information may lead to prolonged training times, thereby enhancing efficiency.

3.4. Motivation

In today's deep learning research and practice, professionals engaged in source code vulnerability detection using LLMs are concerned about issues such as computing power, efficiency, accuracy, and cost. Therefore, this paper focuses on investigating these concerns. To explore these issues, we first evaluate the current mainstream LLMs to study their efficiency and performance in the field of source code vulnerability detection.

Additionally, Due to the distinct features contained in AST and CFG, and their ability to complement each other to include potentially more features possibly relevant to vulnerable patterns [34]. It is hoped that LLMs will demonstrate improved performance on a richer set of feature information. On the other hand, we discuss model efficiency and address it by employing model compression techniques like knowledge distillation and LoRA. Furthermore, we conduct a systematic comparison of these two model compression methods in the domain of source code vulnerability detection, aiming to achieve State-of-the-Art (SOTA) performance from these various approaches.

4. Experimental design

To improve the accuracy and efficiency of source-code level detection, this paper conducts the following experiments, as shown in Figs. 3 and 4.

Firstly, we investigate whether the hybrid framework (VulACLLM) enhances the performance of LLMs in vulnerability detection. The experimental process is as follows: (1) Selection of the Optimal LLM: We partition the collected open-source datasets and conduct experiments in vulnerability detection using four different LLMs. The optimal LLM is selected based on our proposed metrics. (2) Structural Feature Extraction: We process the datasets based on the characteristics of AST and CFG, converting them into vector representations containing structural features that can be input into the LLMs. (3) Validation of the Hybrid Framework (VulACLLM): We validate the effectiveness of our proposed hybrid framework.

To address the efficiency of applying LLMs for vulnerability detection, we also conduct experiments on LLMs compressed using two different model compression techniques. The research process is as follows: (1) Knowledge Distillation: We evaluate the performance of LLMs compressed using knowledge distillation on downstream tasks in vulnerability detection. (2) LoRA Compression: We conduct vulnerability detection using LLMs compressed with the LoRA. Based on various metrics, we compare the LLMs compressed using the two different model compression techniques. This allows us to further select the most suitable LLMs to enhance the performance and efficiency of the hybrid framework (VulACLLM). Additionally, we propose Top-K Efficiency, to quantitatively evaluate the compressed LLMs. This metric will enable practitioners in the field of vulnerability detection to more effectively assess the strengths and weaknesses of LLMs and select models that balance performance and efficiency.

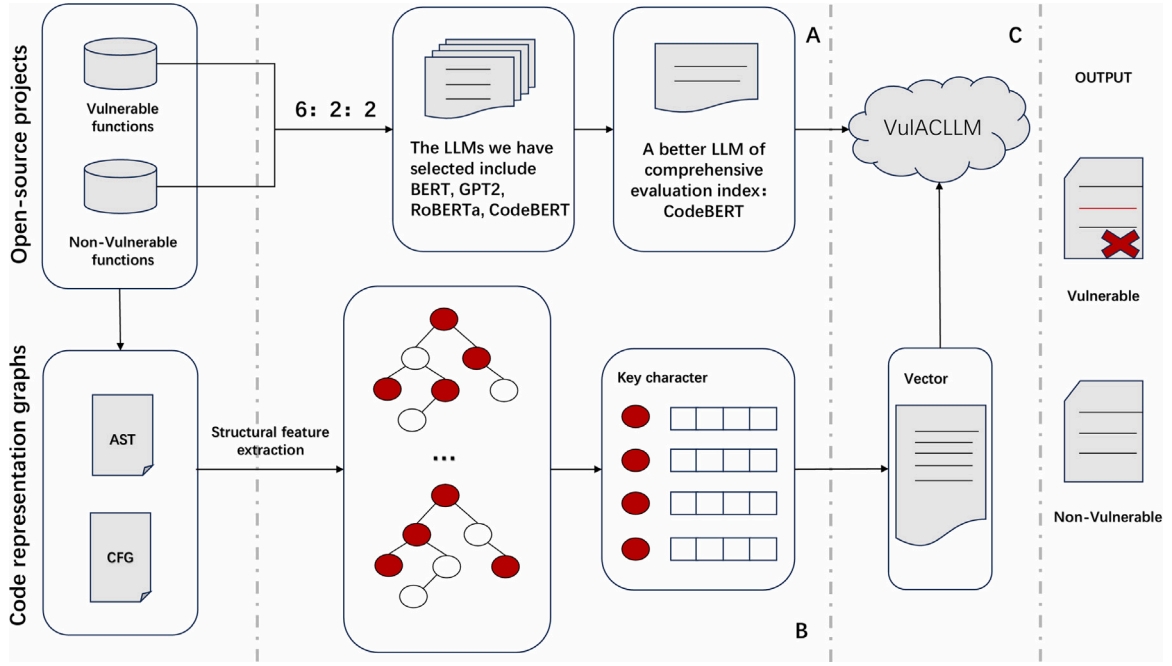


Fig. 3. A schematic diagram of the VulACLLM hybrid framework process.

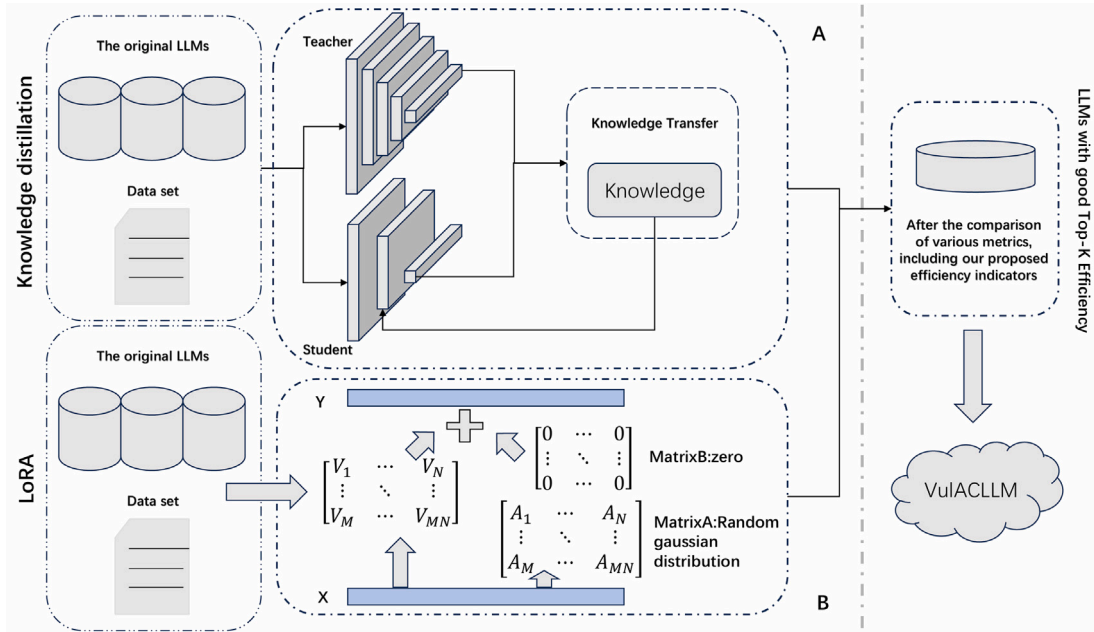


Fig. 4. By comparing the performance of LLMs compressed using Knowledge distillation and LoRA, select the more balanced LLMs to enhance VulACLLM.

4.1. Experiment setup

4.1.1. Data sets: Collection

In this section, we draw from the content of relevant dataset collection efforts. Lin et al. utilized functions as the granularity of defective code, selecting six well-known projects (LibTIFF, LibPNG, FFmpeg, Pidgin, VLC Media Player, Asterisk) [35,36]. They retrieved relevant defects through the NVD and manually identified the functions where defects occurred. Lin et al. also provided detailed descriptions of the data annotation and collection standards within their work.

In this paper, the dataset we utilized collected vulnerable and non-vulnerable functions from 12 open-source software projects written in

the C programming language. These 12 datasets include Asterisk, FFmpeg, Httpd, ImageMagick, LibPNG, LibTIFF, OpenSSL, Pidgin, qemu, samba, VLC, and Xen. The number of functions included in each of these 12 datasets is illustrated in Table 1.

4.1.2. LLMs: Selection

Firstly, the hardware platform for testing performance is the NVIDIA GeForce RTX 3060 12 GB graphics card. This hardware standard was used as a rough basis for the selection of LLMs. Referring to the evolutionary tree of LLMs outlined in “Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond”, we initially focused on models based on the Transformer architecture that appeared before

Table 1

The number of non-vulnerable and vulnerable source-code functions collected from 12 open-source projects.

Projects	# of non_vulnerable functions	# of vulnerable functions
Asterisk	17 620	94
FFmpeg	5549	249
Httpd	3843	57
ImageMagick	2361	344
LibPNG	577	45
LibTIFF	726	123
OpenSSL	7004	159
Pidgin	8549	29
qemu	36 064	143
samba	32 846	26
VLC	6019	44
Xen	8981	670

2021. This preference leaned towards models like the GPT series, constructed from the decoder part of the Transformer architecture, and the BERT series, constructed from the encoder part. The rationale behind this choice is that before the advent of high-parameter models like the GPT3 series, the parameter sizes of models appearing earlier did not significantly exceed the training capacity of consumer-grade graphics cards. For instance, the parameter sizes ranged from 110 million to 340 million for the BERT series, from 150 million to 1.5 billion for the GPT series, and from 118 million to 350 million for the XLNet model.

Lin et al.'s study also assessed the performance and efficiency of PCM (Pre-trained Contextualized Models) and NCM (Non-Contextualized Models) in vulnerability detection, revealing that PCM outperformed NCM in vulnerability detection [3,10]. To investigate which type of LLMs is better suited for vulnerability detection, we selected well-performing pre-trained LLMs as the foundation for this study, such as BERT, GPT-2, as well as variants based on the BERT model like RoBERTa and CodeBERT.

On one hand, the reason for choosing these models is that they are typical models for handling downstream tasks within LLMs. Encoder-only style language models function as BERT and are also autoencoder (AE) language models. Decoder-only style language models function as GPT and are also autoregressive (AR) language models. RoBERTa is an improvement over BERT, incorporating some pre-training details and architecture adjustments, resulting in improved performance. Meanwhile, CodeBERT utilizes both natural language and source code as input during pre-training. On the other hand, in terms of training methods, traditional neural networks typically start training from scratch and require a large amount of labeled vulnerability data for supervised learning, with parameters initialized randomly and updated throughout the training process. In contrast, large language models (LLMs) undergo pre-training on vast amounts of unlabeled data before being fine-tuned for specific tasks. This approach reduces the reliance on extensive labeled data. Given the scarcity of vulnerability data, using LLMs offers significant advantages.

In terms of generalization and versatility, traditional neural networks are often designed for specific tasks and perform well in those areas but tend to have limited generalization ability. For instance, convolutional neural networks (CNNs) are tailored for image recognition, while recurrent neural networks (RNNs) are suited for time-series data. On the other hand, LLMs exhibit strong generalization and versatility. After pre-training on large datasets, they can be fine-tuned for a wide range of downstream tasks (e.g., text classification, question answering, machine translation) with relatively low fine-tuning costs. LLMs leverage universal feature representations, benefiting from transfer learning, and demonstrating enhanced generalization capabilities.

For the aforementioned LLMs, we downloaded the models from Hugging Face. Specifically, for the base models such as BERT, GPT2, RoBERTa, and the variant CodeBERT, we utilized the following models from the Hugging Face platform: bert-base-uncased, gpt2, roberta-base, and codebert-base, respectively. The BERT, GPT2, RoBERTa, and

CodeBERT models selected in this paper have a hidden size of 768, with 12 attention heads and 12 hidden layers. For the selection of distilled models, this paper chose distilled versions based on the aforementioned base models. The distilled models are as follows: DistilBERT, DistilGPT2, DistilCodeBERT, and DistilRoBERTa. The summarized statistics for all the aforementioned information are compiled in Table 2.

4.1.3. Experimental settings

For dataset selection, we continued to employ the vulnerabilities and non-vulnerabilities functions from the 12 open-source projects for training, testing, and evaluation. We utilized Python scripts specifically designed for handling datasets related to software vulnerability detection, named Intelligent-software-vulnerability-detection-main, for data processing.

Firstly, data extraction is initiated by reading files from specified directories (Vulnerable_functions and Non_vulnerable_functions), extracting data from these files, and removing unnecessary characters. Subsequently, a dictionary containing function data, target labels (0 indicating non-vulnerable, 1 indicating vulnerable), indices, and project information is created. Then, vulnerable and non-vulnerable data are combined into one dataset, which is shuffled. The shuffled dataset is split into training, validation, and test sets, with 60% of the data allocated to the training set, and 20% each to the validation and test sets.

We use unified hyperparameters to ensure fair comparison among selected LLMs. The hyperparameters include epoch, block size, train batch size, eval batch size, and learning rate. Table 3 presents the values of the hyperparameters used in our experiments. Finally, the execution phase is carried out, where the four Transformer-based LLMs are utilized as vulnerability predictors to evaluate the models' performance.

4.2. Evaluation metrics

4.2.1. Traditional metrics

To effectively assess the accuracy of the selected LLMs, the dataset was partitioned into three groups — training, validation, and test sets — according to 6:2:2 ratio.

To quantify the predictive performance of selected models, this paper employs commonly used metrics which include True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). They constitute the four key evaluation metrics in the confusion matrix, aiding in a detailed analysis of the LLMs's performance across different categories, particularly in imbalanced datasets. Through these four evaluation metrics, accuracy, recall, precision, and F1-Score can be computed using the following formulas, thereby providing a more comprehensive assessment of the model's performance:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

$$Recall = \frac{TP}{TP + FN} \quad (2)$$

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (4)$$

In the scenario of vulnerability detection, due to the scarcity of vulnerabilities, metrics such as recall, precision, and F1-score can better measure the predictive performance of a model. For a vulnerable function detector, recall measures the ability of the detector to identify all vulnerable functions within the test set. Precision measures the accuracy of the actual vulnerable functions detected by the model, whereas the F1-score balances both recall and precisions.

In addition to the aforementioned metrics, this paper introduces Top-K metrics, tailored for real-world vulnerability detection scenarios. In practice, the number of functions that can be reviewed is limited, as not all code can be audited due to constraints related to time and

Table 2

The Hugging Face model name and settings of LLMs selected for evaluation.

Model	Hugging Face model name	Layers	Attention heads	Hidden size
BERT	bert-base-cased	12	12	768
GPT-2	gpt2	12	12	768
RoBERTa	roberta-base	12	12	768
CodeBERT	codebert-base	12	12	768
DistilBERT	distilbert-base-uncased	6	12	768
DistilGPT2	distilgpt2	6	12	768
DistilRoBERTa	distilroberta-base	6	12	768
DistilCodeBERT	distil_codeBERT	6	12	768

Table 3

Hyperparameter settings applied for all evaluated LLMs.

Block_size	batch_size	Epoch	Learning_rate	Optimizer
400	6	5	0.00002	Adam

resources. Consequently, it is more practical to focus on a subset of functions that most likely to contain vulnerabilities. Therefore, we retrieve K functions and assess their recall and precision within this selected group, which we call Top-K precision (denoted as P@K) and Top-K recall (denoted as R@K). The “K” in Top-K refers to the top K functions ranked by their likelihood of being vulnerable. P@K denotes the proportion of actual vulnerable functions identified by the LLMs in the top K function lists retrieved. R@K is the proportion of actual vulnerable functions among the top K function lists returned that are actually proportion of vulnerable functions found. Due to the limited amount of vulnerable functions in our test set, we set the K ranges from 10 to 300. The two formulas are as follows:

$$P@K = \frac{TP@k}{TP@k + FP@k} \quad (5)$$

$$R@K = \frac{TP@k}{TP@k + FN@k} \quad (6)$$

Where TP@k is the true positive samples which are the actual vulnerable functions identified by the LLMs when retrieving k most likely vulnerable functions. For example, there are 1000 functions in a test set. After prediction, we examine 100 (k = 100) of the total functions which are the most likely to be vulnerable. That is, we retrieve top 100 functions ranked by their probability of being vulnerable and identify how many of these functions are actually vulnerable. Similarly, the FP@k denotes the false vulnerable functions found by the LLMs when returning k most probable vulnerable functions. The FN@k refers to the true vulnerable samples missed by the LLMs when returning k functions.

4.2.2. Top-K Efficiency

The reduction in the compressed model size and the demand for computational resources comes at the cost of performance degradation. Therefore, we propose two metrics for quantitatively analyzing the efficiency of the compressed model. Due to the application of LLMs which require considerable computational resources and time for training, a metric termed “Top-K Efficiency” is proposed to evaluate the efficiency and performance of a model when it is compressed either using KD or LoRA. The formulas for assessing model performance is proposed as follows:

$$T_E = \frac{T_O - T_C}{T_O} \quad (7)$$

$$Top - K_{Efficiency} = \frac{Top - K_C - Top - K_O}{T_E} \quad (8)$$

The T_E metric is utilized to assess the improvement in time efficiency when a model is compressed. Specifically, it measures the degree to which the compressed model’s training speed is enhanced relative to the original model. T_O refers to the training time required for one

epoch of the original LLMs under the specified input conditions. T_C denotes the training time required for one epoch of the compressed LLMs under the specified input conditions. The result of this formula aids in evaluating the improvement in time efficiency of the compressed model.

The Top-K Efficiency formula is employed to assess changes in models’ Top-K metrics (e.g., accuracy, recall, etc.) after each instance of model is compressed by using LoRA or knowledge distillation. As a compressed model has a smaller size on the disk and consumes less video memory, leading to less time for training, validation, and test. We use T_E represents the ratio of time efficiency improvement of the original model to that of the compressed model. $Top - K_O$ refers to the performance of the original model on Top-K metrics, such as accuracy and recall. $Top - K_C$ denotes the performance of the compressed model calculated on these metrics.

This formula aids in evaluating how model performance (measured using Top-K metrics) changes after model compression, considering the efficiency gained by the time reduction of prediction processes. If the Top-K Efficiency is positive, it indicates that the model compression has achieved significant time efficiency and an improvement in performance. If the Top-K Efficiency is negative, it suggests that while the model compression has increased time efficiency, the model’s performance has declined compared to the original model. A higher positive Top-K Efficiency value signifies more notable improvements in both time efficiency and performance. Conversely, a smaller negative Top-K Efficiency value indicates greater performance loss.

By employing this formula, researchers and practitioners can make informed decisions about the trade-offs between model size, detection performance, and efficiency. Compressed models with reduced size offer the possibility for the deployment on consumer GPUs, making them a performance solution.

4.3. Experimental 1: Evaluation of the LLMs

This section summarizes and analyzes the performance of LLMs in the field of source code vulnerability detection. In addition to Experiment 1, two sets of experiments were designed. The Experiment 2 aimed to explore the effectiveness of combining the selected LLMs with our proposed hybrid approach of AST and CFG. The Experiment 3 of experiments systematically compared two different model compression methods, namely “Knowledge Distillation” and LoRA. The objective was to reduce the deployment complexity of LLMs under hardware resource constraints, while also decreasing the time required for LLMs to perform downstream tasks and enhancing overall efficiency.

4.3.1. Preliminary evaluation: LLMs

Neural networks have demonstrated promising results in various fields such as image processing, speech recognition, and natural language processing, distinct from the domain of network security vulnerability detection. In this context, selecting a suitable neural network model has become a primary concern.

This section provides a detailed analysis of several traditional neural network models and non-contextual models (such as Word2Vec, GloVe, etc.) to explain why LLMs were selected for vulnerability detection

Table 4

Top-K precision and Top-K recall of non-contextual models and LLMs(Top-K precision | Top-K recall).

Model category	Model	Top10	Top20	Top30	Top40	Top50	Top100	Top150	Top200	Top250	Top300
Non-contextual models	Word2Vec	50% 1%	60% 3%	60% 5%	65% 7%	68% 9%	64% 16%	52% 20%	50% 25%	48% 30%	43% 33%
	FastTest	70% 2%	70% 4%	75% 6%	78% 8%	82% 10%	64% 16%	55% 21%	47% 24%	41% 26%	38% 29%
	GloVe	30% 1%	40% 2%	35% 3%	33% 3%	32% 4%	33% 8%	30% 11%	28% 14%	27% 17%	24% 18%
	ELMo	60% 2%	65% 3%	75% 6%	80% 8%	78% 10%	72% 18%	65% 25%	55% 28%	49% 31%	45% 34%
LLMs	BERT	40% 1%	50% 3%	57% 4%	60% 6%	58% 7%	68% 17%	69% 26%	74% 37%	74% 47%	71% 54%
	CodeBERT	90% 2%	95% 5%	97% 7%	93% 9%	94% 12%	95% 24%	95% 36%	91% 46%	86% 54%	79% 60%
	GPT2	100% 3%	100% 5%	100% 8%	95% 10%	96% 12%	93% 24%	90% 34%	88% 45%	82% 52%	77% 58%
	RoBERTa	90% 2%	90% 5%	93% 7%	95% 10%	94% 12%	94% 24%	86% 33%	83% 42%	78% 49%	75% 57%

experiments rather than traditional neural network models. Additionally, vulnerability detection experiments were conducted using some non-contextual models on the same dataset for comparison. Meanwhile, comparisons among various LLMs based on the Transformer architecture were conducted after fine-tuning for downstream tasks.

For traditional neural network models such as RNN, the output of the network is dependent on the inputs before the current time step, which results in the network storing previously encountered information and using it for computation and output at the current time step. Due to this attribute, RNN exhibits short-term memory, rendering it susceptible to forgetting earlier information in lengthy sequences. Moreover, when training RNN, issues such as vanishing and exploding gradients frequently arise. This is attributable to the necessity for gradient information to traverse multiple time steps during backpropagation, leading to gradual diminishment or sudden escalation of gradients. As a result, there could be a loss of information over extended sequences, posing challenges for RNN to effectively capture long-term dependencies. Furthermore, RNN processes sequences sequentially, necessitating each time step to await completion of the preceding time step's computation. This yields relatively lower computational efficiency when handling lengthy sequences, thereby impeding the full exploitation of hardware acceleration. As for CNN, it is primarily designed for processing grid-structured data such as images, and may not be as suitable for the sequential data that has long-term dependencies.

As for non-contextual models adjusted based on neural networks (such as Word2Vec, GloVe, etc.), these models generate static word embeddings, which cannot handle polysemy nor dynamically adjust word meanings according to different sentence contexts. Additionally, some models in this category are unable to generate embeddings for words not encountered during training. Furthermore, non-contextual models struggle to capture long-distance dependencies between words within a sentence, being limited to local word co-occurrences. They also lack syntactic and structural information of the language.

Traditional neural network models, especially conventional RNN and some simplistic CNNs, may encounter issues like vanishing or exploding gradients when processing long texts, making it challenging to capture long-term contextual information. LLMs, by employing more sophisticated structures such as Transformers, offer a better understanding of long dependency relationships within text. Additionally, LLMs leverage context-aware word embeddings to dynamically handle polysemy and contextual dependencies. The ability to understand complex semantic relationships is enhanced through the use of self-attention mechanisms. Hence, we opt for testing based on Transformer-based LLMs.

4.3.2. Results

For the LLMs we selected, firstly, we standardized the hyperparameters that might affect model performance. Additionally, we standardized the datasets. The test results are presented in Table 4, which illustrates the outcomes of the evaluation scheme.

Regarding the table, it can be observed that in terms of Top-K accuracy, Non-contextual models significantly underperform LLMs in the field of vulnerability detection. Specifically, non-contextual models achieve a maximum accuracy of only 49% after the top 200 predictions.

In contrast, LLMs, such as GPT-2, demonstrate strong performance, particularly within the top 50 detections. In the range of top 100 to top 300, the CodeBERT model demonstrates superior performance. In the field of vulnerability detection, the CodeBERT model outperforms conventional BERT, GPT2, and RoBERTa models. This could be attributed to several factors: (1) Domain Adaptation: CodeBERT, tailored for processing code texts, leverages a substantial volume of code data during the pre-training stage, thereby demonstrating superior adaptability to code-related tasks. Conversely, models such as BERT, GPT2, and RoBERTa predominantly rely on natural language text data for pre-training, potentially lacking in adequately capturing the nuances and semantics of code. (2) The ability to express language: CodeBERT prioritizes code-specific details such as syntax structures, variable naming conventions, and function call relationships in its design. Consequently, it possesses a heightened capability to accurately depict the meaning and structure of code, thereby enhancing accuracy in tasks such as vulnerability detection. (3) Pre-training data quality: In its pre-training phase, CodeBERT harnesses a considerable corpus of code data, encompassing open-source projects and code snippets. The richness and variety of this dataset render it particularly well-suited for tasks like vulnerability detection, surpassing typical natural language text data in its relevance and applicability. Conversely, models like BERT, GPT2, and RoBERTa undergo pre-training on extensive datasets predominantly centered around natural language text, potentially lacking the depth of coverage needed for code-specific information. (4) Fine-tuning on downstream tasks: CodeBERT could have undergone specialized fine-tuning and optimizations tailored for tasks like vulnerability detection. This might involve integrating supervised signals pertaining to vulnerabilities during the pre-training phase or employing loss functions and evaluation metrics specifically designed for vulnerability detection tasks during fine-tuning. Such strategies play a crucial role in augmenting the model's efficacy in vulnerability detection tasks.

Based on the analysis of the results, Experiment 1 demonstrates the effectiveness of LLMs in the field of vulnerability detection and provides a comprehensive explanation of the superiority of LLMs in this domain.

4.4. Experiment 2: Evaluation of the code analysis diagrams with LLMs

4.4.1. Exploring the combination of AST, CFG, and large language modeling

To investigate whether the integration of LLMs with code analysis graphs can enhance the efficiency of vulnerability detection, we set up the first group of experiments. The first step involved selecting outstanding LLMs from various neural networks and LLMs using the evaluation method. The second step involved experimenting with the integration of AST into LLMs. The third step explored whether integrating AST and CFG into LLMs would improve accuracy.

4.4.2. Results

For the combination of AST, CFG, and LLMs, we employed the same evaluation methods, using Top-K precision, recall, F1-Score, and other metrics to assess the integration of LLMs with code analysis graphs. We set the K ranges from 10 to 200. Because we found that with a K value of 200, VulACLLM sufficiently demonstrates the advantages of our proposed framework. In this study, we conducted experiments in

Table 5

Comparison results of the hybrid framework (VulACLLM), Non-contextual Models and LLMs in terms of the vulnerable class.

Models	Non-contextual models				LLMs			Two frameworks	
	Word2Vec	FastTest	GloVe	ELMo	BERT	GPT2	RoBERTa	AST+LLMs	VulACLLM
F1-Score	0.25	0.24	0.14	0.28	0.61	0.66	0.63	0.66	0.73
Recall	0.5	0.47	0.28	0.55	0.65	0.65	0.62	0.67	0.73
Precision	0.33	0.32	0.19	0.37	0.58	0.67	0.64	0.68	0.74

Table 6

Top-K precision of hybrid framework models.

Hybrid framework model	Top10	Top20	Top30	Top40	Top50	Top100	Top150	Top200
AST+LLMs	90%	95%	83%	83%	82%	67%	71%	66%
AST+CFG+LLMs(VulACLLM)	90%	95%	93%	85%	88%	75%	74%	74%

the field of vulnerability detection and found that our proposed hybrid framework (VulACLLM), shows a significant difference in Top-100 to Top-200 accuracy compared to using LLMs alone for vulnerability detection. However, the hybrid framework surpasses the standalone LLMs in terms of recall and F1-Score, demonstrating the effectiveness of our proposed hybrid framework (VulACLLM). The results are shown in Tables 5 and 6.

For these results, we speculate on several potential reasons. Firstly, AST is typically abstract representations generated after parsing the source code, which may result in the loss of some details and the original code structure. During vulnerability detection, crucial information might be lost due to the abstract representation of AST, making it challenging for models to capture specific vulnerability patterns. Secondly, combining AST with LLMs may lead to inconsistencies in the training data of the model. LLMs are commonly trained directly on source code, while AST represents a highly abstracted form of code. If inconsistencies arise during the conversion or training process between the two, the model may encounter difficulties in accurately learning vulnerability patterns from the source code. Additionally, vulnerability detection often necessitates the identification of intricate vulnerability patterns, which AST may not sufficiently provides detailed and accurate information to capture. LLMs trained at the source code level typically excel at learning fine-grained vulnerability features. Furthermore, AST construction might overlook certain syntactic structures that could serve as crucial features for vulnerability models. LLMs trained at the source code level demonstrate greater proficiency in capturing these specific syntactic structures. Due to the limitations of using AST alone, CFG is incorporated to compensate for the shortcomings of AST.

4.5. Experiment 3: Evaluation of the LoRA and LLMs

4.5.1. Analysis of LoRA and enhanced modeling efficiency

The Low-Rank Adaptation Method (LoRA) is a strategy employed in machine learning and statistics to address high-dimensional data. It facilitates dimensionality reduction by reducing the rank of the data, thereby minimizing redundant information and noise to improve the model's ability to generalize. This method enables quicker and more efficient adaptation of LLMs to specific downstream tasks or alternative domains. Its approach entails integrating a low-rank matrix into the weight matrix of the pre-trained language model. The low-rank adaptation layer initializes with random values and undergoes updates during fine-tuning.

The key steps of the LoRA process are outlined as follows:

Initialization: Commence with a pre-trained language model and incorporate a low-rank adaptation layer into its weight matrix. This layer is represented by a randomly initialized low-rank matrix.

Fine-tuning: Train the model on a new task or domain, updating solely LoRA layer while keeping the weights of the pre-trained model fixed. This enables the model to effectively learn task-specific information without modifying other weights.

Prediction: Utilize the adapted model to generate predictions on new, unseen data from the target task or domain.

By concentrating the adaptation process on the low-rank matrix, LoRA achieves more efficient fine-tuning, thereby reducing computational and memory overhead in comparison to comprehensive fine-tuning approaches. Consequently, LoRA enables quicker and more effective fine-tuning of LLMs while conserving computational resources, energy, and time. Furthermore, LoRA can be applied to various LLMs like GPT and BERT, and readily adapted to diverse tasks and domains. Ultimately, LoRA can deliver competitive performance, rendering it a compelling alternative for numerous applications.

Improving model efficiency is a key challenge in the field of machine learning, particularly in the context of large-scale data and complex models. Prior to this, efforts have been made by many researchers in the following areas: (1) Model Pruning: By eliminating redundant neurons, connections, or subnetworks, the model's size is reduced. Abdullah Salama et al. pruned neural networks by removing entire filters and neurons based on global thresholds of the entire neural network, demonstrating not only good performance on related tasks but also ease of implementation [37]. (2) Quantization: The principle behind quantization is to convert model parameters and activation values from high-precision representations (e.g., 32-bit floating-point numbers) to low-precision representations (e.g., 8-bit integers), thus reducing the model's memory footprint and computational cost. To reduce computation and memory costs, Jiwei Yang et al. represented weights and activations as differentiable non-linear functions, with the quantization function expressed as a linear combination of several Sigmoid functions with learnable biases and scales. Extensive experiments in image classification and object detection tasks showed that their quantized networks outperformed state-of-the-art methods [38]. (3) Model Distillation: The principle of this method to improve model efficiency involves using the output of a larger model (teacher model) as labels to train a smaller model (student model). This helps the student model learn the knowledge of the teacher model, thereby reducing the model's size. Linfeng Zhang et al. proposed a new distillation method on this basis: modifying the original neural network into a multi-head neural network and performing knowledge distillation on these classifiers. This approach achieved significant improvements in model accuracy, acceleration, and compression [39].

4.5.2. Performance comparison with the LoRA + LLMs and the distilled LLMs

The various efficiency-enhancing methods mentioned in the previous section, such as model distillation, used in vulnerability detection in cybersecurity, have matured, such as distillation of BERT models, quantization, and pruning of certain neural networks. LoRA was tested on nine NLU tasks in the GLUE (General Language Understanding Evaluation) benchmark. LoRA was used to freeze the original weights while learning the rank decomposition matrix to reduce the trainable parameters. The nine tasks involve natural language inference, text entailment, sentiment analysis, semantic similarity, and other tasks. On the GLUE test results, LoRA outperformed several other adaptation methods, including adapters, prefix-tuning, and fine-tuning. However, research tasks using LoRA in vulnerability detection are relatively

Table 7

Top-K precision and Top-K recall of Distillation models, LoRA models, and LLMs(Top-K precision | Top-K recall)

Model category	Model	Top10	Top20	Top30	Top40	Top50	Top100	Top150	Top200	Top250	Top300
LLMs	BERT	40% 1%	50% 3%	57% 4%	60% 6%	58% 7%	68% 17%	69% 26%	74% 37%	74% 47%	71% 54%
	CodeBERT	90% 2%	95% 5%	97% 7%	93% 9%	94% 12%	95% 24%	95% 36%	91% 46%	86% 54%	79% 60%
	GPT2	100% 3%	100% 5%	100% 8%	95% 10%	96% 12%	93% 24%	90% 34%	88% 45%	82% 52%	77% 58%
	RoBERTa	90% 2%	90% 5%	93% 7%	95% 10%	94% 12%	94% 24%	86% 33%	83% 42%	78% 49%	75% 57%
Distillation models	DistilBERT	90% 2%	95% 5%	97% 7%	93% 9%	96% 12%	89% 23%	89% 34%	86% 44%	81% 51%	73% 55%
	DistilCodeBERT	90% 2%	95% 5%	93% 7%	95% 10%	96% 12%	97% 25%	93% 35%	85% 43%	79% 50%	73% 55%
	DistilGPT2	100% 3%	95% 5%	97% 7%	95% 10%	92% 12%	93% 24%	85% 32%	89% 45%	83% 53%	76% 58%
	DistilRoBERTa	100% 3%	85% 4%	80% 6%	85% 9%	84% 11%	83% 24%	83% 32%	84% 43%	81% 51%	78% 59%
LoRA models	LoRABERT	80% 2%	85% 4%	87% 7%	88% 9%	80% 10%	68% 17%	60% 23%	59% 30%	53% 34%	50% 38%
	LoRAGPT2	90% 2%	95% 5%	93% 7%	85% 9%	74% 9%	73% 18%	62% 24%	61% 31%	57% 36%	53% 40%
	LoRARoBERTa	80% 2%	65% 3%	53% 4%	63% 6%	66% 8%	68% 17%	65% 25%	58% 29%	54% 34%	50% 38%

Table 8

Comparison results of model performance metrics regarding the vulnerable class.

Model	F1-Score	Recall	Time (Min/Epoch)	Size on disk
BERT DistilBERT LoRABERT	0.61 0.63 0.27	0.65 0.57 0.78	70 40 20	1.7 GB 838 MB 416 MB
GPT2 DistilGPT2 LoRAGPT2	0.66 0.6 0.25	0.65 0.49 0.77	80 50 20	3.41 GB 2.49 GB 5.47 MB
RoBERTa DistilRoBERTa LoRARoBERTa	0.63 0.65 0.14	0.62 0.65 0.82	70 50 20	2.62 GB 1.83 GB 4.89 MB

scarce. Therefore, we propose to use LoRA on the excellent LLMs selected in Experiment One to explore whether similar results to those achieved by LoRA in the nine tasks of GLUE can be achieved in the field of cybersecurity. This aims to achieve efficiency improvements and resource savings in the domain of cybersecurity amidst the prevalence of LLMs in deep learning.

The experimental setup is as follows: the selected LLMs are fine-tuned through LoRA training, and then their performance is compared with distilled models using the same dataset and hyperparameter settings. We continue to employ the same evaluation methodology, utilizing Top-K accuracy to assess the performance of both approaches. Both models trained with LoRA and distilled models were subjected to statistical analysis in terms of F1-Score, Recall, and time, aiming to investigate the comparative advantages of LLMs trained using LoRA.

The experimental results are shown in the Table 7. The indicators of the compression model are shown in Table 8.

From the results, it can be inferred that the distilled models perform well in the field of vulnerability detection, even surpassing the undistilled LLMs. Possible reasons for this performance could be as follows: Firstly, through the distillation process, the student model is able to comprehend the decision-making process of the teacher model to a certain extent, thereby enhancing its generalization ability through knowledge transfer. Additionally, through distillation, the student model learns the most crucial knowledge from the teacher model, avoiding excessive fitting to the noise in the training data, thus reducing the risk of overfitting.

For the LLMs fine-tuned with LoRA, there is a slight decrease in performance in terms of Top-K accuracy and F1-Score metrics, but they outperform both the distilled models and the original models in terms of recall and time. The possible reason is that LoRA typically compresses model parameters to reduce computational and storage costs. In this process, some important information may discard due to compression, leading to a decrease in the model's representational capacity. This loss of information could impact the model's performance. Additionally, LoRA may reduce the complexity of the model, failing to capture all the intricate patterns and semantics present in the original model. Moreover, LoRA may result in the amplification of certain features [33], making the model unable to accurately capture important features in the input data. Furthermore, models trained with the LoRA may not adapt well to the specific requirements of certain tasks, especially when the task features do not match those of the dataset used during the original model training [33]. Indeed, these factors could significantly influence the model's performance. However, in many cases, the classification performance is not the only metric that deserves considerations. The effectiveness of fine-tuning, the adaptability of the

model to new tasks or domains, the conservation of computational resources, and the competitiveness of the achieved performance are all considerations that directly impact the model's utility and efficacy in real-world applications. Therefore, careful attention to these factors is essential to ensure optimal performance and reliable outcomes in various tasks and domains.

LoRA can enhance its ability to detect vulnerabilities by emphasizing more important features based on specific downstream tasks such as vulnerability detection. According to our experimental results in Table 8, it can be observed that this emphasis on key features can positively boost the performance of the model in tasks of vulnerability detection. (Models compressed using the LoRA method can detect more vulnerabilities than those compressed using knowledge distillation). We hypothesize that models fine-tuned using the LoRA approach will reduce model complexity and parameter count, thereby decreasing sensitivity to noise and reducing overfitting. And the LoRA approach may "Certain features that were not specifically mentioned are amplified". This makes the model more focused on identifying the minority class which is the vulnerability class. Therefore, employing LoRA as a model compression method can enhance its ability to detect vulnerability categories, and consequently result in improved recall compared with their distilled counterpart.

After fine-tuning with the LoRA method, the LLMs experiences a significant reduction in the time spent on model training. We believe several factors may contribute to this effect. Firstly, the LoRA method reduces the model's size by compressing its parameters, which in turn reduces the computational and storage requirements during training. Secondly, this approach may decrease the model's complexity, leading to a corresponding reduction in computational complexity during training. This implies that fewer computational resources are needed during the training process, potentially enhancing model parallelism and thus reducing training time.

4.5.3. Top-K Efficiency of LoRA-compressed LLMs and distilled LLMs

In this paper, the performance statistics of LLMs compressed using two model compression methods in the source-code level vulnerability detection tasks are analyzed for Top-K performance, with the value of k being set to 10–300. The results are shown in the Table 9.

Table 9 presents the Top-K Efficiency of three original LLMs after being compressed using different methods. Based on a comprehensive analysis of T_E and Top-K Efficiency for models compressed using the two methods, we find that models compressed using knowledge distillation demonstrate improvements in both time efficiency and performance. In contrast, models compressed using the LoRA method

Table 9

Top-K Efficiency of LLMs after applying KD and LoRA model compression techniques.

Compression techniques	Model	Top10	Top20	Top30	Top40	Top50	Top100	Top150	Top200	Top250	Top300
KD	DistilRoBERTa	0.35	-0.17	-0.45	-0.35	-0.35	-0.35	-0.10	0.03	0.10	0.10
	DistilBERT	1.16	1.05	0.93	0.78	0.88	0.49	0.47	0.28	0.16	0.05
	DistilGPT2	0	-0.13	-0.08	0	-0.11	0	-0.13	0.03	0.03	-0.03
LoRA	LoRARoBERTa	-0.14	-0.35	-0.56	-0.45	-0.39	-0.37	-0.30	-0.35	-0.34	-0.35
	LoRABERT	0.56	0.49	0.42	0.39	0.31	0	-0.13	-0.21	-0.30	-0.30
	LoRAGPT2	-0.13	-0.06	-0.09	-0.13	-0.29	-0.27	-0.37	-0.36	-0.33	-0.32

sacrifices some performance to achieve significant gains in time efficiency.

Analyzing the data in the table, models compressed using knowledge distillation generally show positive Top-K Efficiency values, indicating enhanced performance and time efficiency. For example, when $K = 10$, the Top-K Efficiency for the BERT model is 1.16, suggesting notable improvements in both time efficiency and performance after compression. However, for RoBERTa and GPT2 models compressed with knowledge distillation, the Top-K Efficiency values are frequently negative, and time efficiency improvements are not significant, making this compression method less recommended for RoBERTa and GPT2.

For models compressed using the LoRA method, the BERT model often shows positive Top-K Efficiency values, along with substantial reductions in time. When $K = 10$, the Top-K Efficiency is 0.56, making this method more recommended for the BERT model. For RoBERTa and GPT2 models, there is no significant performance loss, and the demands on hardware resources and training time are noticeably reduced, achieving a balance between model performance and time efficiency. However, the choice between knowledge distillation and LoRA compression methods should be based on the specific circumstances.

Additionally, based on the data in the Table 9, we can conclude that larger Top-K Efficiency values indicate that the compressed model has significantly improved performance and time efficiency. This also implies a reduction in the size of the compressed model, making it applicable to a wider range of scenarios and addressing the challenge of deploying LLMs due to hardware resource constraints. Furthermore, these models can save time in training LLMs for downstream tasks. Models compressed using LoRA also demonstrate good performance in the field of source code vulnerability detection, validating the effectiveness and feasibility of fine-tuning LLMs with LoRA. Thus, the appropriate compression method can be selected based on the Top-K efficiency value and integrated into the proposed VulACLLM framework.

5. Conclusion

In the field of vulnerability detection, we first benchmarked several mainstream LLMs, testing their performance in source code vulnerability detection. The test results indicated that the bimodal pre-trained model CodeBERT, which utilizes both programming language (PL) and natural language (NL), performed well under the Top-K metric, demonstrating the advantage of this pre-trained model in the domain of source code vulnerability detection. To enhance the performance of LLMs in the field of vulnerability detection, a hybrid approach combining AST & CFG with high-performing LLMs was explored through experimental investigation. Our findings indicate that the proposed hybrid framework (VulACLLM) surpasses the performance of original LLMs in metrics such as recall rate and F1-Score.

Due to the extensive computational resources required by these LLMs, which hindered the widespread application of vulnerability detection schemes based on them, we implemented several compression schemes based on LLMs. We systematically compared the performance of these models in vulnerability detection after adopting techniques such as knowledge distillation and LoRA-based compression. We found that distilled models could achieve or even surpass the performance of the original LLMs, as evidenced by their performance in terms of F1-Score and Top-K metrics. Additionally, models fine-tuned with LoRA

surpassed the original LLMs in terms of recall and exhibited significant savings in time for downstream tasks in LLMs training, thereby achieving the goal of resource efficiency. We have also introduced the Top-K Efficiency metric, aimed at assessing the performance and efficiency of compressed models to achieve a balance between the two. By selecting LLMs that strike a better balance, our proposed vulnerability detection framework (VulACLLM) can be further enhanced in terms of performance and efficiency, striving for SOTA capabilities.

In future work, more modern datasets will be employed to explore whether LLMs and the proposed framework are better suited for more modern open source projects (written in C).

CRedit authorship contribution statement

Jianing Liu: Writing – original draft, Validation, Software, Methodology, Formal analysis, Conceptualization. **Guanjun Lin:** Writing – review & editing, Software, Resources. **Huan Mei:** Software, Resources, Methodology. **Fan Yang:** Writing – review & editing, Supervision. **Yonghang Tai:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (62365017, 62062069, 62062070, 62005235), Yunnan Outstanding Youth Fund, China (202301AW070001) and the Fujian Natural Science Foundation, China Project (2021J011131).

Data availability

Data will be made available on request.

References

- [1] Guo J, Long K, Yang K, Jiang K, Lu L, Wang C. A novel prediction method for vulnerability outbreak trend. *Comput Electr Eng* 2023;108:108743.
- [2] Anium S. DoS attacks, triad and privacy: Software exposures in Microsoft, Apple and Google. In: 2020 first international conference of smart systems and emerging technologies. IEEE; 2020, p. 53–8.
- [3] Lin G, Wen S, Han Q-L, Zhang J, Xiang Y. Software vulnerability detection using deep neural networks: a survey. *Proc IEEE* 2020;108(10):1825–48.
- [4] Warren M, Štítelis D, Laurinaitis M. The impact of Russian cyber attackers within the Ukraine situation. *J Inf Warfare* 2023;22(1).
- [5] Juutilainen J. Cyber warfare: A part of the Russo-Ukrainian War in 2022. 2022.
- [6] Floridi L, Chiriatti M. GPT-3: Its nature, scope, limits, and consequences. *Minds Mach* 2020;30:681–94.
- [7] Devlin J, Chang M-W, Lee K, Toutanova K. Bert: Pre-training of deep bidirectional transformers for language understanding. 2018, arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805).
- [8] Yang Z, Dai Z, Yang Y, Carbonell J, Salakhutdinov RR, Le QV. Xlnet: Generalized autoregressive pretraining for language understanding. *Adv Neural Inf Process Syst* 2019;32.
- [9] Touvron H, Lavril T, Izacard G, Martinet X, Lachaux M-A, Lacroix T, et al. Llama: Open and efficient foundation language models. 2023, arXiv preprint [arXiv:2302.13971](https://arxiv.org/abs/2302.13971).

- [10] Lin G, Jia H, Wu D. Distilled and contextualized neural models benchmarked for vulnerable function detection. *Mathematics* 2022;10(23):4482.
- [11] Lin G, Xiao W, Zhang J, Xiang Y. Deep learning-based vulnerable function detection: A benchmark. In: *Information and communications security: 21st international conference, ICICS 2019, Beijing, China, December 15–17, 2019, revised selected papers* 21. Springer; 2020, p. 219–32.
- [12] Lin G, Zhang J, Luo W, Pan L, Xiang Y. POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In: *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 2017, p. 2539–41.
- [13] Mei H, Lin G, Fang D, Zhang J. Detecting vulnerabilities in IoT software: New hybrid model and comprehensive data analysis. *J Inf Secur Appl* 2023;74:103467.
- [14] Yuan X, Lin G, Tai Y, Zhang J. Deep neural embedding for software vulnerability discovery: Comparison and optimization. *Secur Commun Netw* 2022;2022:1–12.
- [15] Zeng P, Lin G, Pan L, Tai Y, Zhang J. Software vulnerability analysis and discovery using deep learning techniques: A survey. *IEEE Access* 2020;8:197158–72.
- [16] Jadidi Z, Muthukkumarasamy V, Sithirasanen E, Sheikhan M. Flow-based anomaly detection using neural network optimized with GSA algorithm. In: *2013 IEEE 33rd international conference on distributed computing systems workshops*. IEEE; 2013, p. 76–81.
- [17] Meng Q, Zhang B, Feng C, Tang C. Detecting buffer boundary violations based on SVM. In: *2016 3rd international conference on information science and control engineering*. IEEE; 2016, p. 313–6.
- [18] Li W, Ge J, Dai G. Detecting malware for android platform: An svm-based approach. In: *2015 IEEE 2nd international conference on cyber security and cloud computing*. IEEE; 2015, p. 464–9.
- [19] An JH, Wang Z, Joe I. A CNN-based automatic vulnerability detection. *EURASIP J Wireless Commun Networking* 2023;2023(1):41.
- [20] Guo J, Wang Z, Li H, Xue Y. Detecting vulnerability in source code using CNN and LSTM network. *Soft Comput* 2023;27(2):1131–41.
- [21] Li X, Wang L, Xin Y, Yang Y, Tang Q, Chen Y. Automated software vulnerability detection based on hybrid neural network. *Appl Sci* 2021;11(7):3201.
- [22] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. *Adv Neural Inf Process Syst* 2017;30.
- [23] Kim S, Choi J, Ahmed ME, Nepal S, Kim H. Vuldebert: A vulnerability detection system using bert. In: *2022 IEEE international symposium on software reliability engineering workshops*. IEEE; 2022, p. 69–74.
- [24] Huang W, Lin S, Chen L. Bbvd: A bert-based method for vulnerability detection. *Int J Adv Comput Sci Appl* 2022;13(12).
- [25] Noever D. Can large language models find and fix vulnerable software? 2023, arXiv preprint arXiv:2308.10345.
- [26] Tian Z, Tian B, Lv J, Chen Y, Chen L. Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding. *Expert Syst Appl* 2024;238:121865.
- [27] Zhou M, Chen J, Liu Y, Ackah-Arthur H, Chen S, Zhang Q, et al. A method for software vulnerability detection based on improved control flow graph. *Wuhan Univ J Nat Sci* 2019;24(2):149–60.
- [28] Strubell E, Ganesh A, McCallum A. Energy and policy considerations for deep learning in NLP. 2019, arXiv preprint arXiv:1906.02243.
- [29] Tian Z, Tian B, Lv J. Combining AST segmentation and deep semantic extraction for function level vulnerability detection. In: *The international conference on natural computation, fuzzy systems and knowledge discovery*. Springer; 2022, p. 93–100.
- [30] Marashdih AW, Zaaba ZF, Suwais K. AST-based LSTM neural network for predicting input validation vulnerabilities. In: *International conference on cybersecurity, cybercrimes, and smart emerging technologies*. Springer; 2022, p. 111–22.
- [31] Feng H, Fu X, Sun H, Wang H, Zhang Y. Efficient vulnerability detection based on abstract syntax tree and deep learning. In: *IEEE INFOCOM 2020-IEEE conference on computer communications workshops*. IEEE; 2020, p. 722–7.
- [32] Guo Z, Zhang C, Fan Y, Tian Y, Zhang C, Chawla NV. Boosting graph neural networks via adaptive knowledge distillation. In: *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 6. 2023, p. 7793–801.
- [33] Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, et al. Lora: Low-rank adaptation of large language models. 2021, arXiv preprint arXiv:2106.09685.
- [34] Yamaguchi F, Golde N, Arp D, Rieck K. Modeling and discovering vulnerabilities with code property graphs. In: *2014 IEEE symposium on security and privacy*. IEEE; 2014, p. 590–604.
- [35] Lin G, Zhang J, Luo W, Pan L, Xiang Y, De Vel O, et al. Cross-project transfer representation learning for vulnerable function discovery. *IEEE Trans Ind Inf* 2018;14(7):3289–97.
- [36] Lin G, Xiao W, Zhang LY, Gao S, Tai Y, Zhang J. Deep neural-based vulnerability discovery demystified: data, model and performance. *Neural Comput Appl* 2021;33(20):13287–300.
- [37] Salama A, Ostapenko O, Klein T, Nabi M. Pruning at a glance: Global neural pruning for model compression. 2019, arXiv preprint arXiv:1912.00200.
- [38] Yang J, Shen X, Xing J, Tian X, Li H, Deng B, et al. Quantization networks. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, p. 7308–16.
- [39] Zhang L, Bao C, Ma K. Self-distillation: Towards efficient and compact neural networks. *IEEE Trans Pattern Anal Mach Intell* 2021;44(8):4388–403.